

ASSIGNMENT-6.1

NAME:E.NIKHIL

H.NO:2303A51847

B-13

Task Description #1 (AI-Based Code Completion for Loops)

Task: Use an AI code completion tool to generate a loop-based program.

Prompt:

“Generate Python code to print all even numbers between 1 and N using a loop.”

Expected Output:

- AI-generated loop logic.
- Identification of loop type used (for or while).
- Validation with sample inputs.

PROMPT:

#Generate Python code to print all even numbers between 1 and N using a loop.

CODE:

```
6.1.py > ...
1  #Generate Python code to print all even numbers between 1 and N using a loop.
2  N = int(input("Enter a number: "))
3  for i in range(1, N + 1):
4      if i % 2 == 0:
5          print(i)
6      else:
7          continue
8
```

OUTPUT:

```
Enter a number: 15
2
4
6
8
10
12
14
```

Justification:

Using AI to generate loop-based code improves learning efficiency by showing a working example instantly. Students can focus more on understanding how even numbers are identified using conditions rather than spending time debugging loop syntax.

Task Description #2 (AI-Based Code Completion for Loop with Conditionals)

Task: Use an AI code completion tool to combine loops and conditionals.

Prompt:

"Generate Python code to count how many numbers in a list are even and odd."

Expected Output:

- AI-generated code using loop and if condition.
- Correct count validation.
- Explanation of logic flow.

PROMPT:

#Generate Python code to count how many numbers in a list are even and odd.

CODE:

```
#Generate Python code to count how many numbers in a list are even and odd.
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
even_count = 0
odd_count = 0

for num in numbers:
    if num % 2 == 0:
        even_count += 1
    else:
        odd_count += 1

print(f"Even numbers: {even_count}")
print(f"Odd numbers: {odd_count}")
```

OUTPUT:

```
Even numbers: 5
Odd numbers: 5
```

Justification:

AI assistance improves code clarity by producing well-structured and readable logic. This helps students understand how control flow works when loops and if-else statements are used together in real programs.

Task Description #3 (AI-Based Code Completion for Class Attributes Validation)

Task: Use an AI tool to complete a Python class that validates user input.

Prompt:

"Generate a Python class User that validates age and email using conditional statements."

Expected Output:

- AI-generated class with validation logic.
- Verification of condition handling.
- Test cases for valid and invalid inputs.

PROMPT:

#Generate a Python class User that validates age and email using conditional statements.

CODE:

```
#Generate a Python class User that validates age and email using conditional statements.
class User:
    def __init__(self, age, email):
        self.age = age
        self.email = email
    def validate_age(self):
        if self.age < 0:
            return "Age cannot be negative."
        elif self.age > 150:
            return "Age is too high."
        else:
            return "Age is valid."
    def validate_email(self):
        if "@" not in self.email:
            return "Invalid email address."
        else:
            return "Email is valid."
# Example usage
user = User(25, "user@example.com")
print(user.validate_age())
print(user.validate_email())
```

OUTPUT:

```
Age is valid.  
Email is valid.
```

Justification :

By generating object-oriented validation code, AI helps learners understand how real-world rules can be enforced using classes and conditionals, promoting better programming practices.

Task Description #4 (AI-Based Code Completion for Classes)

Task: Use an AI code completion tool to generate a Python class for managing student details.

Prompt:

“Generate a Python class Student with attributes (name, roll number, marks) and methods to calculate total and average marks.”

Expected Output:

- AI-generated class code.
- Verification of correctness and completeness of class structure.
- Minor manual improvements (if needed) with justification.

PROMPT:

#Generate a Python class Student with attributes (name, roll number, marks) and methods to calculate total and average marks.

CODE:

```

6.1.py > Student > calculate_average
1  #Generate a Python class Student with attributes (name, roll number,marks) and methods to c
2  class Student:
3      def __init__(self, name, roll_number, marks):
4          self.name = name
5          self.roll_number = roll_number
6          self.marks = marks
7
8      def calculate_total(self):
9          return sum(self.marks)
10
11     def calculate_average(self):
12         total = self.calculate_total()
13         return total / len(self.marks) if self.marks else 0
14
15     # Example usage
16     student = Student("Alice", 101, [85, 90, 78, 92])
17     print(f"Total Marks: {student.calculate_total()}")
18     print(f"Average Marks: {student.calculate_average()}")

```

OUTPUT:

```

Total Marks: 345
Average Marks: 86.25

```

Justification :

The AI-generated class provides a clear example of how data (attributes) and behavior (methods) are combined in object-oriented programming, making it easier for students to grasp OOP concepts.

Task Description 5 (AI-Assisted Code Completion Review)

Task: Use an AI tool to generate a complete Python program using classes, loops, and conditionals together.

Prompt:

"Generate a Python program for a simple bank account system using class, loops, and conditional statements."

Expected Output:

- Complete AI-generated program.
- Identification of strengths and limitations of AI suggestions.
- Reflection on how AI assisted coding productivity

PROMPT:

#Generate a Python program for a simple bank account system using class, loops, and conditional statements.

CODE:

```
6.1.py > BankAccount > _init_
1 #Generate a Python program for a simple bank account system using class, loops, and conditional statements.
2 class BankAccount:
3     def __init__(self, account_number, account_holder, initial_balance=0):
4         self.account_number = account_number
5         self.account_holder = account_holder
6         self.balance = initial_balance
7     def deposit(self, amount):
8         if amount > 0:
9             self.balance += amount
10            print(f"Deposited ${amount}. New balance: ${self.balance}")
11        else:
12            print("Invalid deposit amount.")
13    def withdraw(self, amount):
14        if 0 < amount <= self.balance:
15            self.balance -= amount
16            print(f"Withdrew ${amount}. New balance: ${self.balance}")
17        else:
18            print("Invalid withdrawal amount or insufficient funds.")
19    def get_account_info(self):
20        return f"Account Number: {self.account_number}, Account Holder: {self.account_holder}, Balance: ${self.balance}"
21 account = BankAccount("123456789", "John Doe", 1000)
22 print(account.get_account_info())
23 account.deposit(500)
24 account.withdraw(200)
25 print(account.get_account_info())
```

OUTPUT:

```
Account Number: 123456789, Account Holder: John Doe, Balance: $1000
Deposited $500. New balance: $1500
Withdrew $200. New balance: $1300
Account Number: 123456789, Account Holder: John Doe, Balance: $1300
```

Justification :

Reviewing AI-generated programs helps identify strengths and limitations of automated suggestions. This encourages critical thinking and helps developers understand when manual improvements are necessary, leading to better overall code quality.